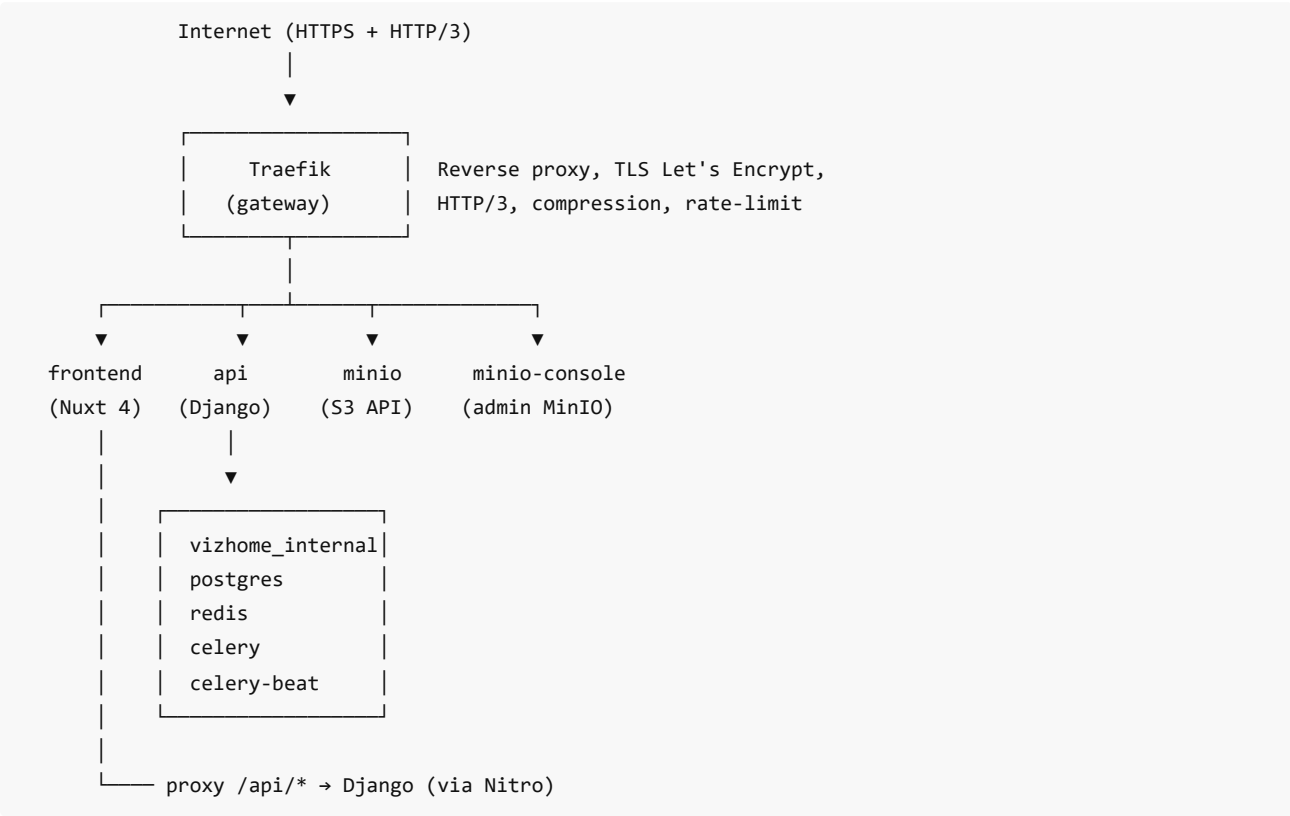


Documentation Installateur — VizHome

Guide complet de mise en œuvre de l'infrastructure et déploiement en production de la solution VizHome (backend Django + frontend Nuxt 4 + services annexes).

Vue d'ensemble de l'architecture



Domaines et services exposés

Domaine	Service	Description
vizhome.fr + www.vizhome.fr	Frontend Nuxt	Application web principale
api.vizhome.fr	Backend Django	API REST
cdn.vizhome.fr	MinIO (S3)	Assets publics (renders, modèles 3D)
minio.vizhome.fr	MinIO Console	Administration du stockage
traefik.vizhome.fr	Traefik Dashboard	Supervision du proxy

Prérequis serveur

Matériel / VM

Ressource	Minimum	Recommandé
CPU	2 vCPU	4 vCPU
RAM	4 Go	8 Go

Disque	40 Go SSD	100 Go SSD
OS	Linux 64-bit (Debian/Ubuntu/Rocky)	Ubuntu 22.04 LTS

Logiciels à installer sur le serveur

```
# Docker Engine (Ubuntu)
curl -fsSL https://get.docker.com | bash
sudo usermod -aG docker $USER

# Vérifier Compose v2
docker compose version

# Doit afficher Docker Compose version v2.x.x
```

Réseau / Firewall

Les ports suivants doivent être ouverts en entrée :

Port	Protocole	Usage
80	TCP	HTTP (redirection vers HTTPS)
443	TCP	HTTPS
443	UDP	HTTP/3 (QUIC)

Les ports internes (5432, 6379, 9000, etc.) ne doivent **pas** être exposés publiquement.

DNS — Enregistrements A requis

Avant de démarrer, créer les enregistrements DNS A suivants pointant vers l'IP de votre serveur :

```
vizhome.fr      A  <IP_SERVEUR>
www.vizhome.fr  A  <IP_SERVEUR>
api.vizhome.fr  A  <IP_SERVEUR>
cdn.vizhome.fr  A  <IP_SERVEUR>
minio.vizhome.fr A  <IP_SERVEUR>
traefik.vizhome.fr A <IP_SERVEUR>
```

La propagation DNS peut prendre jusqu'à 24h. Traefik ne peut générer les certificats TLS qu'une fois les DNS résolus.

Déploiement — Étape par étape

Étape 1 — Préparer le serveur

```
ssh root@<IP_SERVEUR>

# Répertoire de déploiement
mkdir -p /opt/vizhome
cd /opt/vizhome

# Créer le network Docker partagé Traefik (une seule fois)
docker network create vizhome_proxy
```

Étape 2 — Cloner les dépôts

```
cd /opt/vizhome

git clone https://github.com/VizHome/backend-vizhome.git
git clone https://github.com/VizHome/frontend-vizhome.git
```

Étape 3 — Configurer le backend

```
cd /opt/vizhome/backend-vizhome
cp .env.prod.example .env.prod
nano .env.prod
```

Variables obligatoires

Variable	Valeur	Comment générer
DJANGO_SECRET_KEY	Chaîne aléatoire 64 chars	python3 -c "import secrets; print(secrets.token_urlsafe(64))"
DJANGO_DEBUG	False	—
POSTGRES_PASSWORD	Mot de passe fort	openssl rand -base64 32
MINIO_S3_ACCESS_KEY	Identifiant MinIO	Libre choix (ex: vizhome_prod)
MINIO_S3_SECRET_KEY	Clé secrète MinIO	openssl rand -base64 32
ACME_EMAIL	Email admin	Pour les alertes Let's Encrypt
API_HOST	api.vizhome.fr	—
FRONTEND_HOST	vizhome.fr	—
MINIO_PUBLIC_HOST	cdn.vizhome.fr	—
MINIO_CONSOLE_HOST	minio.vizhome.fr	—
TRAEFIK_DASHBOARD_HOST	traefik.vizhome.fr	—
TRAEFIK_DASHBOARD_AUTH	Hash Basic Auth	htpasswd -nbB admin "MOT_DE_PASSE_FORT"

Variables services tiers (optionnels mais recommandés)

Variable	Source	Fonctionnalité
GEMINI_API_KEY	https://aistudio.google.com/apikey	Génération IA (renders)
STRIPE_LIVE_SECRET_KEY	dashboard.stripe.com	Paiements
STRIPE_LIVE_PUBLISHABLE_KEY	dashboard.stripe.com	Paiements (frontend)
STRIPE_WEBHOOK_SECRET	Stripe Dashboard → Webhooks	Synchronisation abonnements
GOOGLE_OAUTH_CLIENT_ID	Google Cloud Console	Connexion Google
GOOGLE_OAUTH_CLIENT_SECRET	Google Cloud Console	Connexion Google
GITHUB_OAUTH_CLIENT_ID	github.com/settings/developers	Connexion GitHub

GITHUB_OAUTH_CLIENT_SECRET	github.com/settings/developers	Connexion GitHub
SENTRY_DSN	sentry.io	Monitoring erreurs (prod)
EMAIL_HOST	Votre fournisseur SMTP	Emails transactionnels
EMAIL_HOST_USER	—	—
EMAIL_HOST_PASSWORD	—	—

Guide détaillé pour chaque clé : [SETUP KEYS.md](#)

Configuration email en production

```
# Exemple avec SendGrid
EMAIL_BACKEND=django.core.mail.backends.smtp.EmailBackend
EMAIL_HOST=smtp.sendgrid.net
EMAIL_PORT=587
EMAIL_HOST_USER=apikey
EMAIL_HOST_PASSWORD=SG.xxxxxxxxxx
DEFAULT_FROM_EMAIL=noreply@vizhome.fr
```

Étape 4 — Démarrer le backend

```
cd /opt/vizhome/backend-vizhome
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d
```

Au premier démarrage, l'entrypoint exécute automatiquement :

1. Attente que PostgreSQL et Redis soient prêts (healthcheck)
2. `migrate` — Application des migrations de base de données
3. `collectstatic` — Collecte des fichiers statiques Django admin
4. `seed_forum_categories` — Création des catégories forum par défaut
5. `setup_stripe_products` — Création des produits Stripe (si configuré)
6. `setup_webhook_endpoint` — Enregistrement du webhook Stripe (si configuré)
7. Démarrage de **Gunicorn** (4 workers)

Aucune commande manuelle n'est requise. Tout est orchestré par `manage.py bootstrap`.

Étape 5 — Configurer le frontend

```
cd /opt/vizhome/frontend-vizhome
cp .env.example .env.prod
nano .env.prod
```

Variables clés du frontend :

```
NEXT_PUBLIC_API_BASE_URL=https://api.vizhome.fr/api/v1
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live...
NEXT_PUBLIC_GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com
```

Démarrer le frontend :

```
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d
```

Étape 6 — Vérifications post-déploiement

```
# Healthcheck backend
curl -fsS https://api.vizhome.fr/health/ready
# Attendu : {"status":"ok","checks":{"postgres":"ok","redis":"ok"}}

# Frontend
curl -fsS -o /dev/null -w "%{http_code}\n" https://vizhome.fr
# Attendu : 200

# Statut des containers
docker compose -f docker-compose.prod.yml --env-file .env.prod ps

# Logs Traefik (certificats TLS)
docker compose -f docker-compose.prod.yml --env-file .env.prod logs traefik
```

Étape 7 — Créer le superutilisateur admin

```
docker compose -f docker-compose.prod.yml --env-file .env.prod exec api \
python manage.py createsuperuser
```

Étape 8 — Configurer le webhook Stripe (production)

Dans le **Dashboard Stripe** → Webhooks → **Add endpoint** :

- **URL** : `https://api.vizhome.fr/webhooks/stripe/webhook/`
- **Events à écouter** :
 - `customer.subscription.created`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
 - `checkout.session.completed`
 - `invoice.payment_succeeded`
 - `invoice.payment_failed`

Copier le **Signing secret** (`whsec_XXX`) dans `.env.prod` :

```
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXXXXXXXX
```

Puis recréer le container api pour recharger l'env :

```
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --force-recreate api
```

Configuration des tâches planifiées (Celery Beat)

Via l'admin Django (`https://api.vizhome.fr/admin/` → Django Celery Beat → Periodic Tasks) :

Tâche	Crontab	Description
<code>accounts.reset_monthly_render_counters</code>	<code>0 0 1 * *</code>	Remet à zéro les compteurs de rendus IA (1er du mois)

admin_panel.snapshot_metrics	5 0 * * *	Snapshot quotidien des métriques admin
forum.cleanup_orphan_uploads	0 3 * * *	Nettoyage des images forum orphelines

Architecture des réseaux Docker

```
vizhome_proxy (external)
├─ traefik
├─ api (Django)
├─ minio
└─ frontend (Nuxt)

vizhome_internal (bridge – non exposé)
├─ postgres
├─ redis
├─ celery
└─ celery-beat
```

Configuration Traefik

Fichiers de configuration

```
traefik/
├─ traefik.yml      # Config statique (entrypoints, ACME, métriques)
└─ dynamic/        # Config dynamique (hot-reload)
    ├─ middlewares.yml # Security headers, rate-limit, compression
    └─ tls.yml        # Options TLS 1.3
```

Middlewares actifs

Middleware	Effet
security-headers	HSTS preload, X-Frame-Options, COOP/CORP
compress	Brotli + gzip
rate-limit-global	100 req/s, burst 200, par IP
rate-limit-strict	5 req/min (login, register, contact)
redirect-www-to-apex	Canonicalisation <code>www.vizhome.fr</code> → <code>vizhome.fr</code>

TLS

- Provider : **Let's Encrypt** (ACME)
- Challenge : HTTP-01 + TLS-ALPN-01
- Protocoles : TLS 1.2 minimum, TLS 1.3 préféré
- HTTP/3 (QUIC) activé sur le port 443/UDP

Mise à jour

```
cd /opt/vizhome/backend-vizhome
git pull origin main
```

```
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
```

L'entrypoint applique automatiquement les nouvelles migrations au démarrage.

Mise à jour du frontend

```
cd /opt/vizhome/frontend-vizhome
git pull origin main
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
```

Rollback

```
# Lister les commits récents
git log --oneline -10

# Revenir à un commit précédent
git checkout <commit-hash>
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
```

Attention : si le commit cible n'inclut pas la dernière migration, exécuter manuellement `python manage.py migrate <app> <migration_précédente>`.

Checklist de sécurité pré-production

- ☐ DJANGO_DEBUG=False
- ☐ DJANGO_SECRET_KEY généré aléatoirement, jamais versionné
- ☐ Mots de passe Postgres + MinIO ≥ 32 caractères
- ☐ HTTPS forcé via Traefik (HSTS activé)
- ☐ Webhook Stripe avec signature validée
- ☐ CORS limité à `vizhome.fr` uniquement
- ☐ Console MinIO accessible uniquement via VPN ou IP allowlist
- ☐ Ports internes (5432, 6379, 9000) non exposés publiquement
- ☐ Backups testés (restore vérifié au moins une fois)
- ☐ Monitoring Sentry configuré

Support

Canal	Adresse
Email technique	dev@vizhome.fr
Sécurité	security@vizhome.fr
GitHub Issues	https://github.com/VizHome/backend-vizhome/issues